#15

CS 0447
Introduction to
Computer Programming

Multicycle Design

Original slides by: David Wilkinson
Modified with bits from: Bruce Childers, and Jarrett Billingsley

Luís Oliveira

Fall 2020

Multicycle

Multicycle Design

- Simply put: let instructions take more than one clock cycle to complete.



Single-cycle

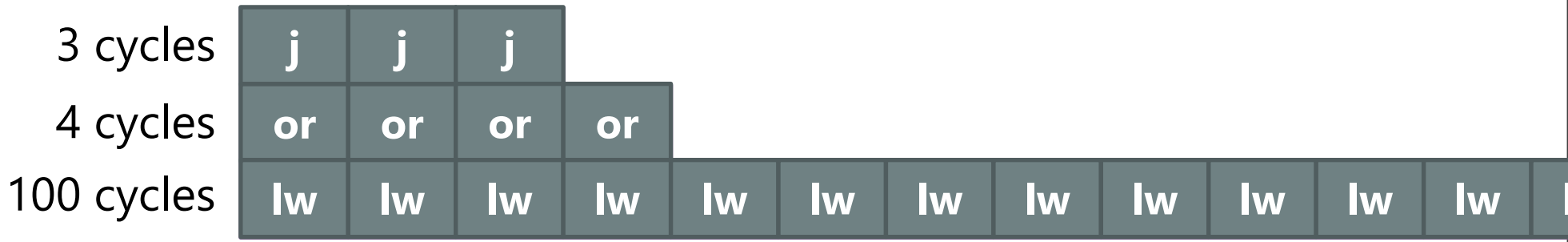


PHOTO: https://www.straitstimes.com/asia/east-asia/china-scrambles-to-tame-rental-bike-chaos

Multicycle?

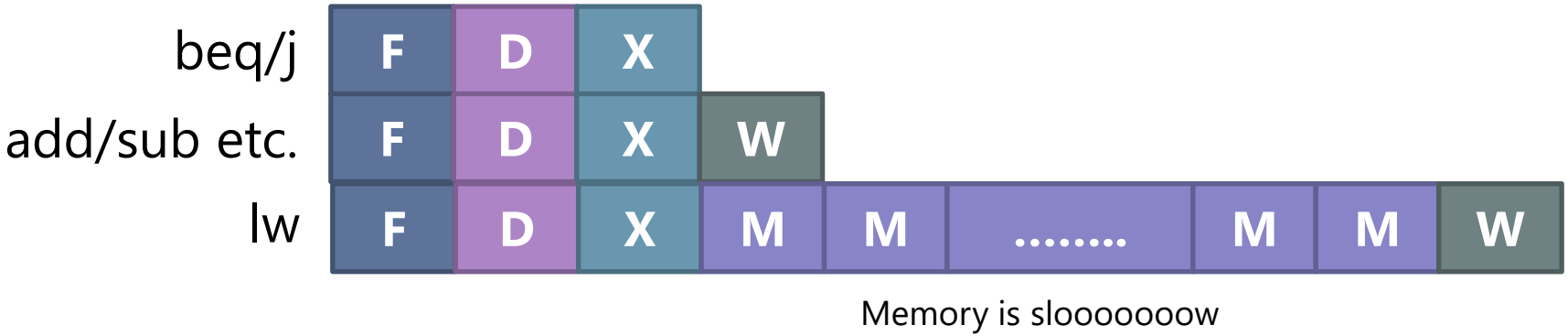
Chop chop

- Not all instructions take the same amount of time, so…
- Make different instructions take **different amounts of time!**
  - And by that, we mean ***different numbers of clock cycles***



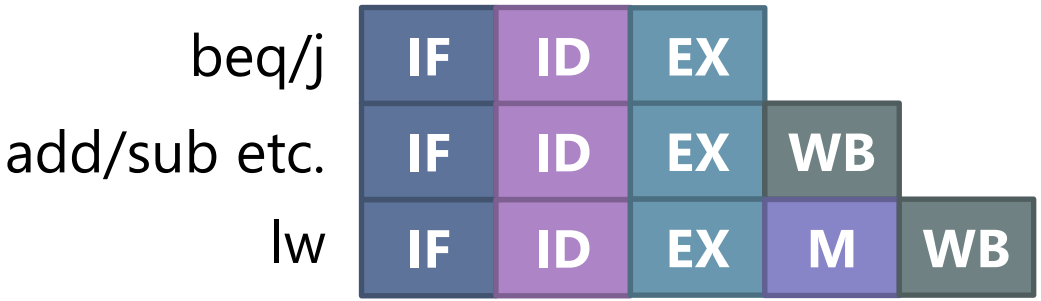
The instructions' steps

- Why would some instructions take less time?
- Recall the **five phases of execution?**
- All instructions have IF, ID, EX, but *only some* write to memory/regs
  - We're managing their complexity with respect to time.



The instructions' steps

- Why would some instructions take less time?
- Recall the **five phases of execution?**
- All instructions have IF, ID, EX, but *only some* write to memory/regs
  - We're managing their complexity with respect to time.

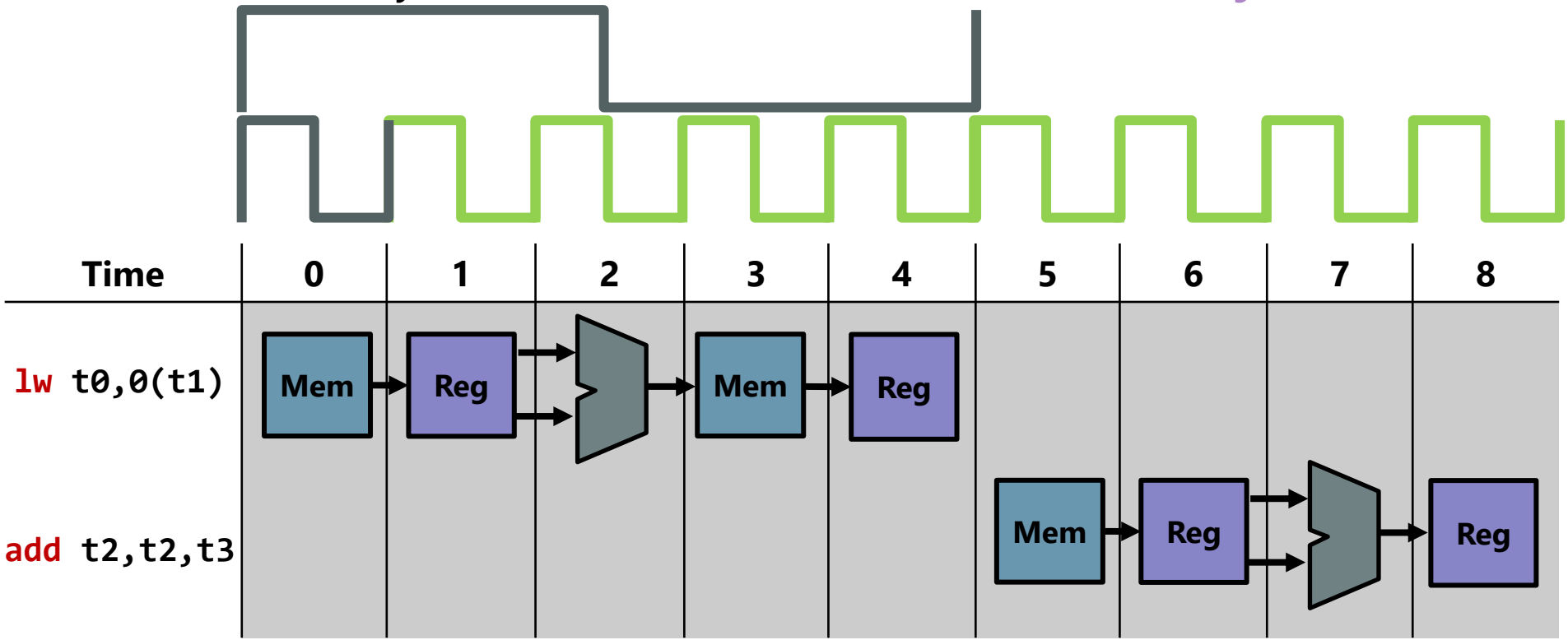


(let's just say lw is 5 cycles :)

Multi-cycle CPU

Calculate clock to accommodate a single phase.

- Chop instructions and make the **clock faster**
- Less time wasted by faster instructions! → **Reduces latency**



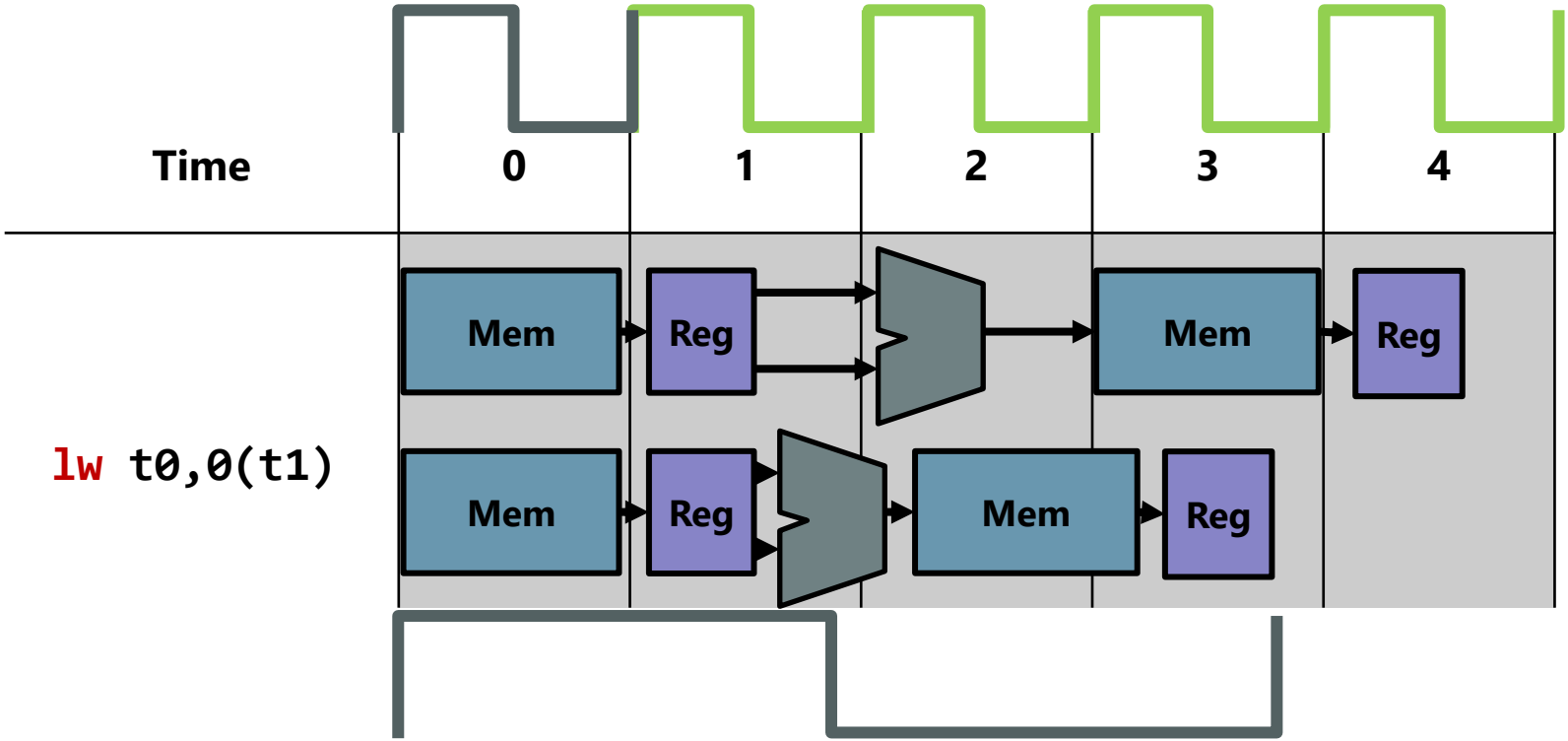
Not all stages are the same ☹

In the multi-cycle design:

Slowest stage **limits** the rate

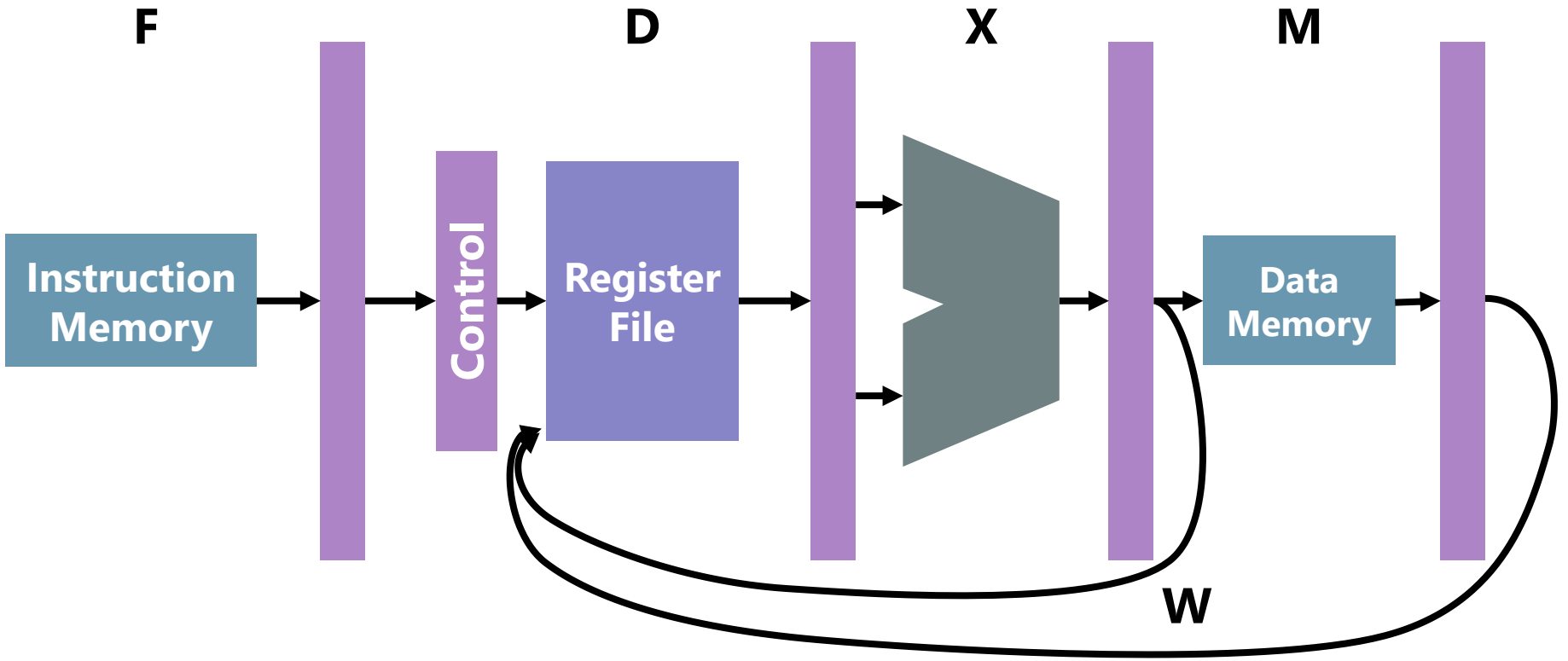
Balanced stages are desired

E.g. split the memory operation into multiple clock cycles



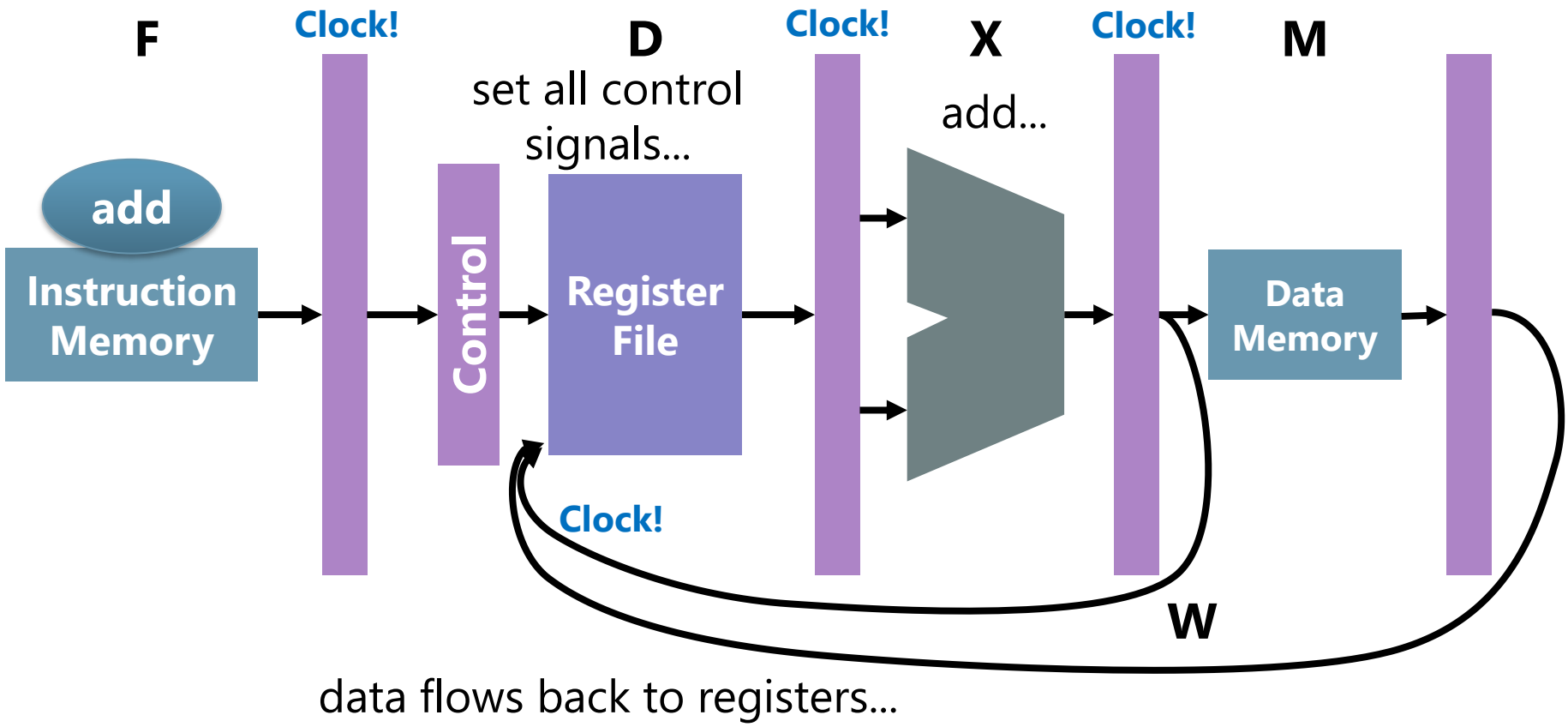
The multicycle datapath from a bird's-eye view

- Each phase of execution has its own **functional unit**
- between phases, we insert **registers** to hold onto the data for the next phase. (*Recall registers ➔ sequential logic*)



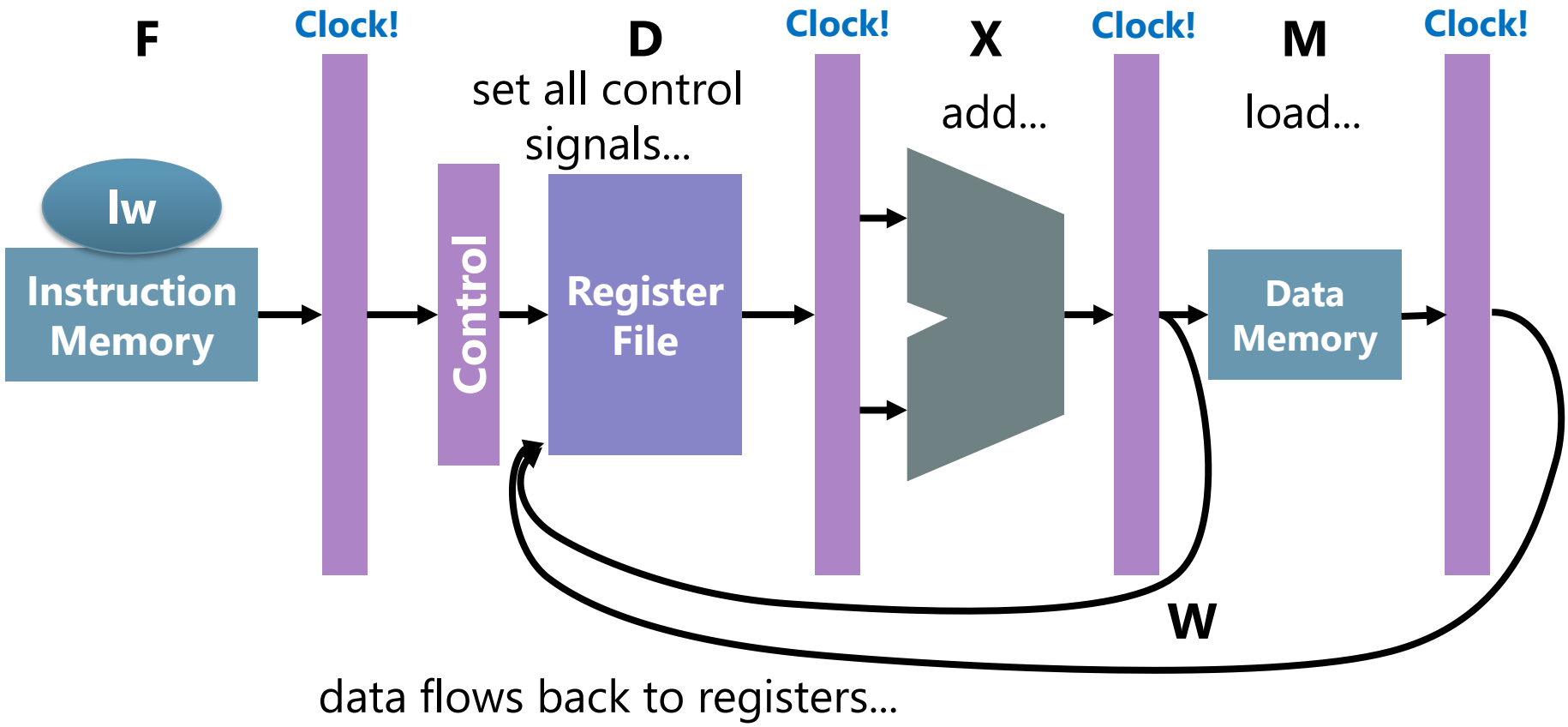
Watching an add (animated)

- Let's watch an **add** instruction flow through the datapath!



Watching a lw (animated)

- Let's watch a **lw** instruction flow through the datapath!

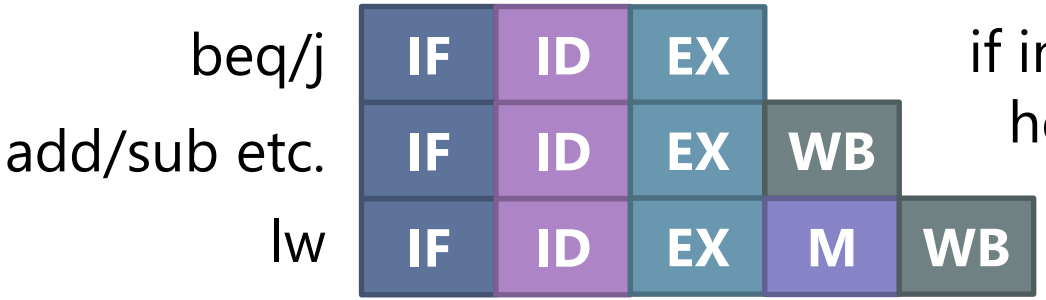


CPI (and IPC)

- **CPI** (Cycles Per Instruction) measures the **average** number of cycles it takes to complete **one instruction**
- **IPC** (instructions per cycle) is its reciprocal
  - **multi-issue** CPUs can execute multiple instructions in one clock cycle! WOAH 8O
- So, what's the CPI for the **single-cycle** implementation?
  - uh, 1.
  - By, yanno, definition.
- What about for a **multicycle** implementation?
  - ……????? hmmm

So what the heck has this bought us?

- Let's say our clock cycle time decreased from 5ns to 1ns!
  - that's from 200 MHz to 1 GHz! :D
- ...buuut our CPI (cycles per instruction) increased a lot.
  - with the single-cycle datapath, CPI was always 1.
  - now the CPI is... well... uh... variable?



if instructions vary in length, how do we calculate CPI?

Calculating Average CPI

- Every program is different, and every program has a different **instruction mix** – how many of each kind of instruction it uses
- Let's say we have a program where 60% of the instructions are ALU, 20% are branches, 15% are loads, and 5% are stores.

	ALU		Branches		Loads		Stores
%	60%		20%		15%		5%
Cycles	4		3		5		4
CPI	2.4	+	0.6	+	0.75	+	0.2

2. Now **sum** the CPIs

= 3.95

1. for each category, **multiply** the **proportion** (percentage) by the **number of cycles** for that category to get the **per-category CPI**

this is the **Average CPI** for **THIS** program. different mixes give different CPIs!

The performance equation

- If we have *n* instructions, and each instruction takes *CPI* cycles, and each cycle takes *t* seconds, how long does it take to execute all the instructions?

$$\text{Total time} = n \text{ instructions} \times \frac{CPI \text{ cycles}}{\text{instruction}} \times \frac{t \text{ seconds}}{\text{cycle}}$$

$$= n \times CPI \times t \text{ seconds}$$

or in English, it's the product of the instruction count, the CPI, and the length of one clock cycle

So how much better is it?!??!?

- Say we execute 500 mega (500×10^6) instructions
- For the single-cycle datapath:
  - CPI = 1
  - cycle time = 5ns (5×10^{-9} s)
  - total time = n × CPI × cycle time
    - = $(500 \times 10^6) \times (1) \times (5 \times 10^{-9})$
    - = **2.5 seconds.**
- For the multicycle datapath:
  - CPI = 3.95 **(much higher!)** (again, this CPI is only for *this* program)
  - cycle time = 1ns **(much lower!)**
  - total time = $(500 \times 10^6) \times (3.95) \times (1 \times 10^{-9})$
  - = **1.975 seconds!**

How does it look like?

Multicycle Datapath + Control (no control flow)

- A potential example of a multicycle control for MIPS:
  - Additional registers and multiplexers hold and then direct temporary data.

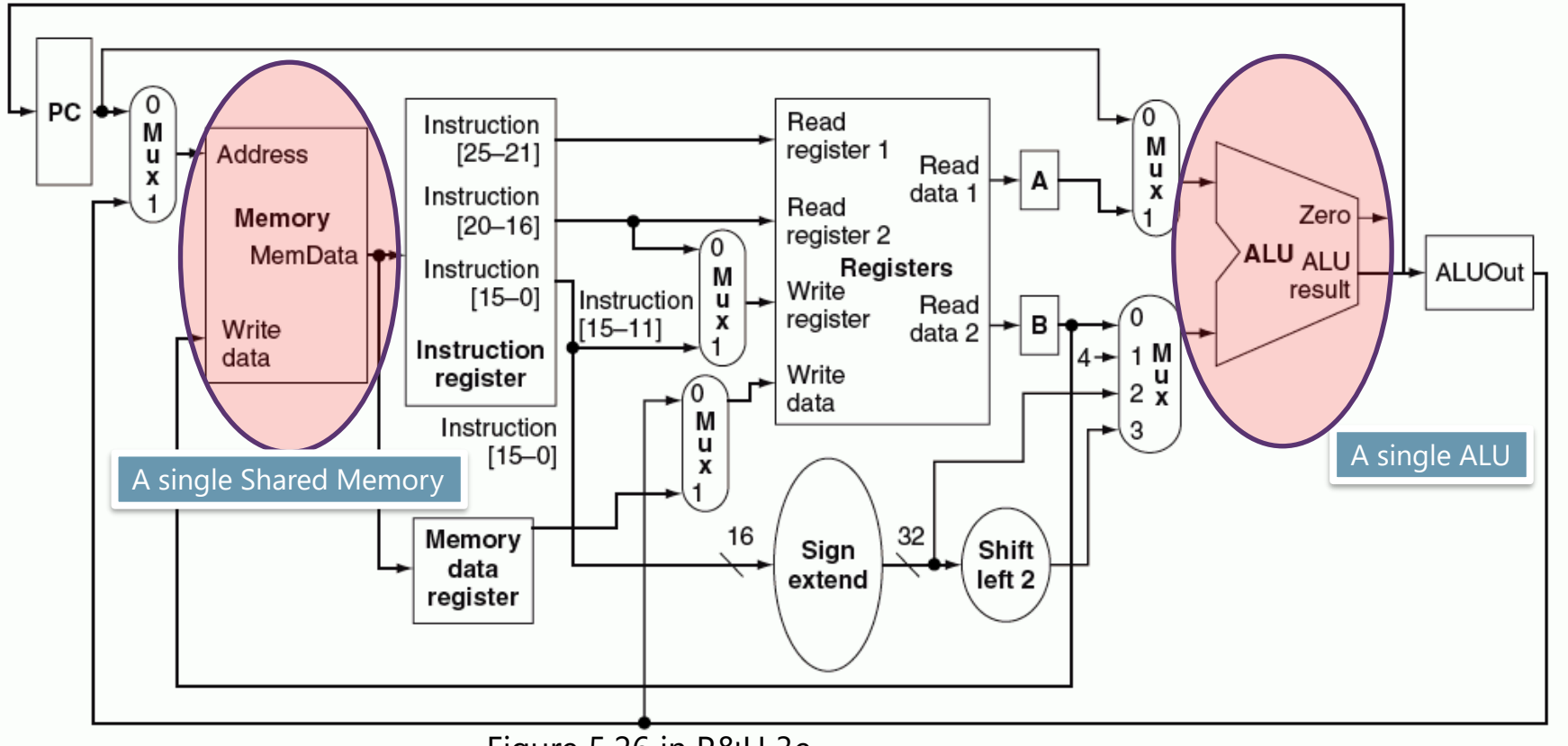


Figure 5.26 in P&H 3e

Multicycle Datapath + Control (no control flow)

- A potential example of a multicycle control for MIPS:
  - Additional registers and multiplexers hold and then direct temporary data.

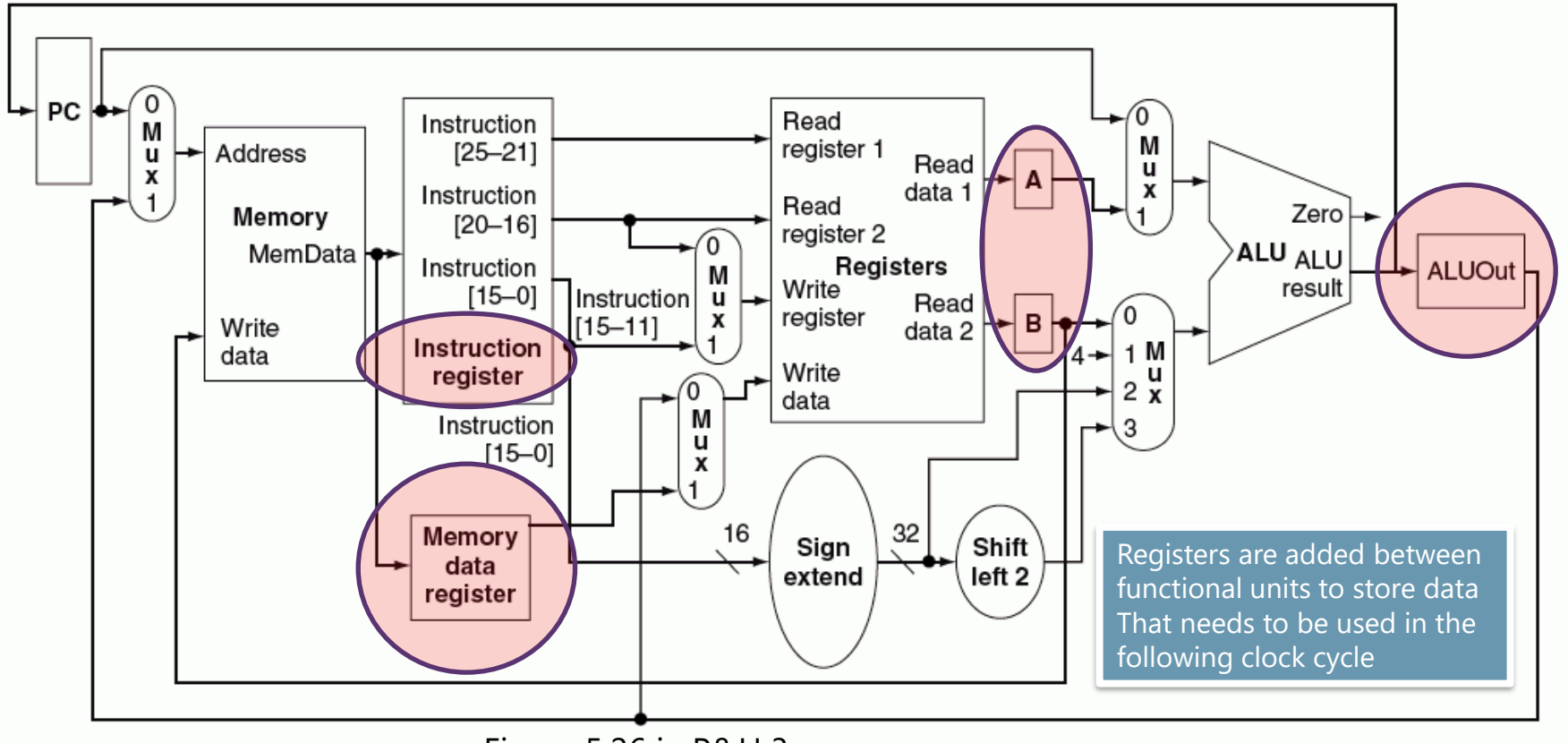


Figure 5.26 in P&H 3e

Multicycle Datapath + Control (no control flow)

- A potential example of a multicycle control for MIPS:
  - Additional registers and multiplexers hold and then direct temporary data.

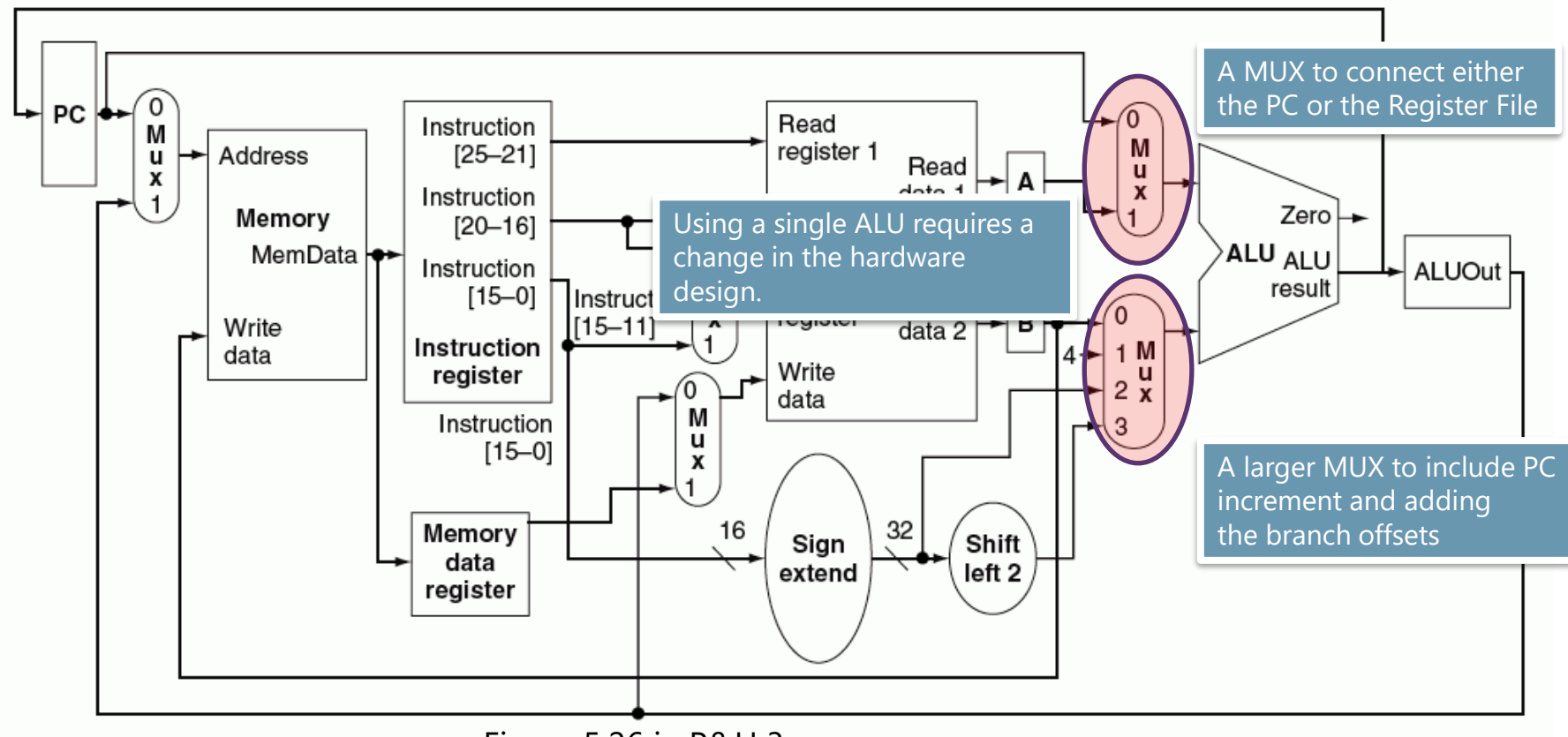


Figure 5.26 in P&H 3e

Multicycle Datapath + Control (with control flow)

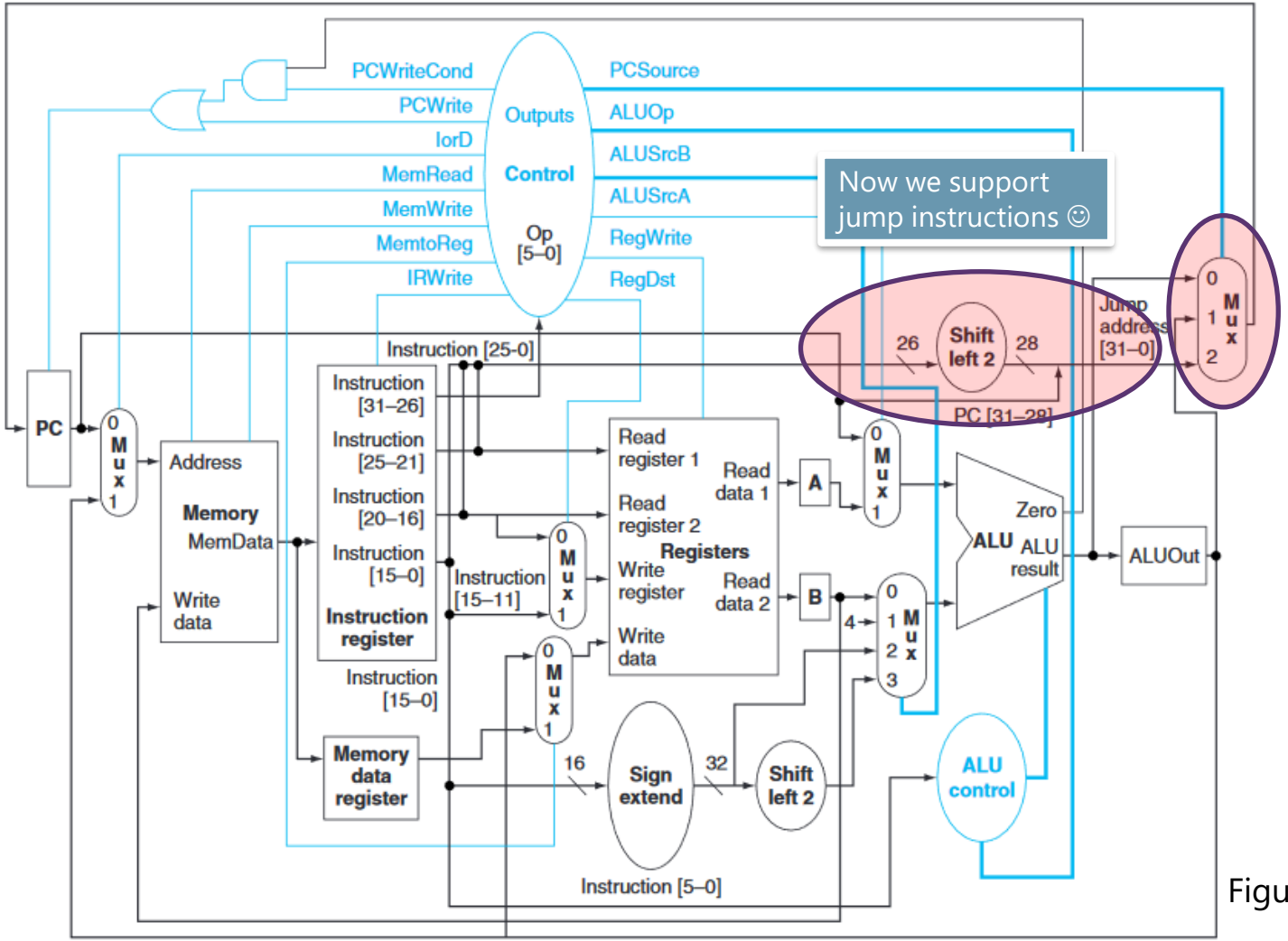


Figure 5.28 in P&H 3e

Multicycle Datapath + Control (with control flow)

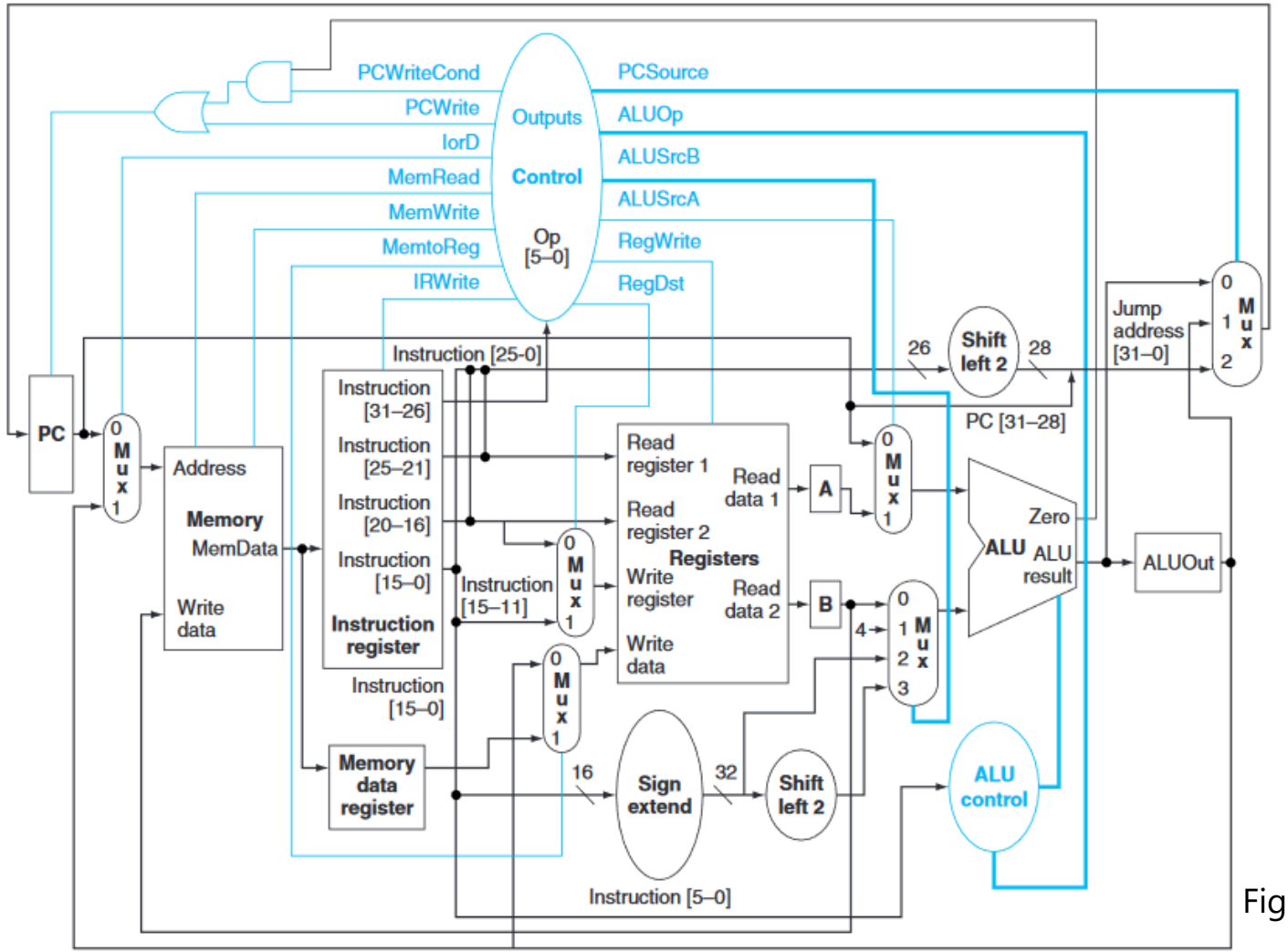


Figure 5.28 in P&H 3e

Signals

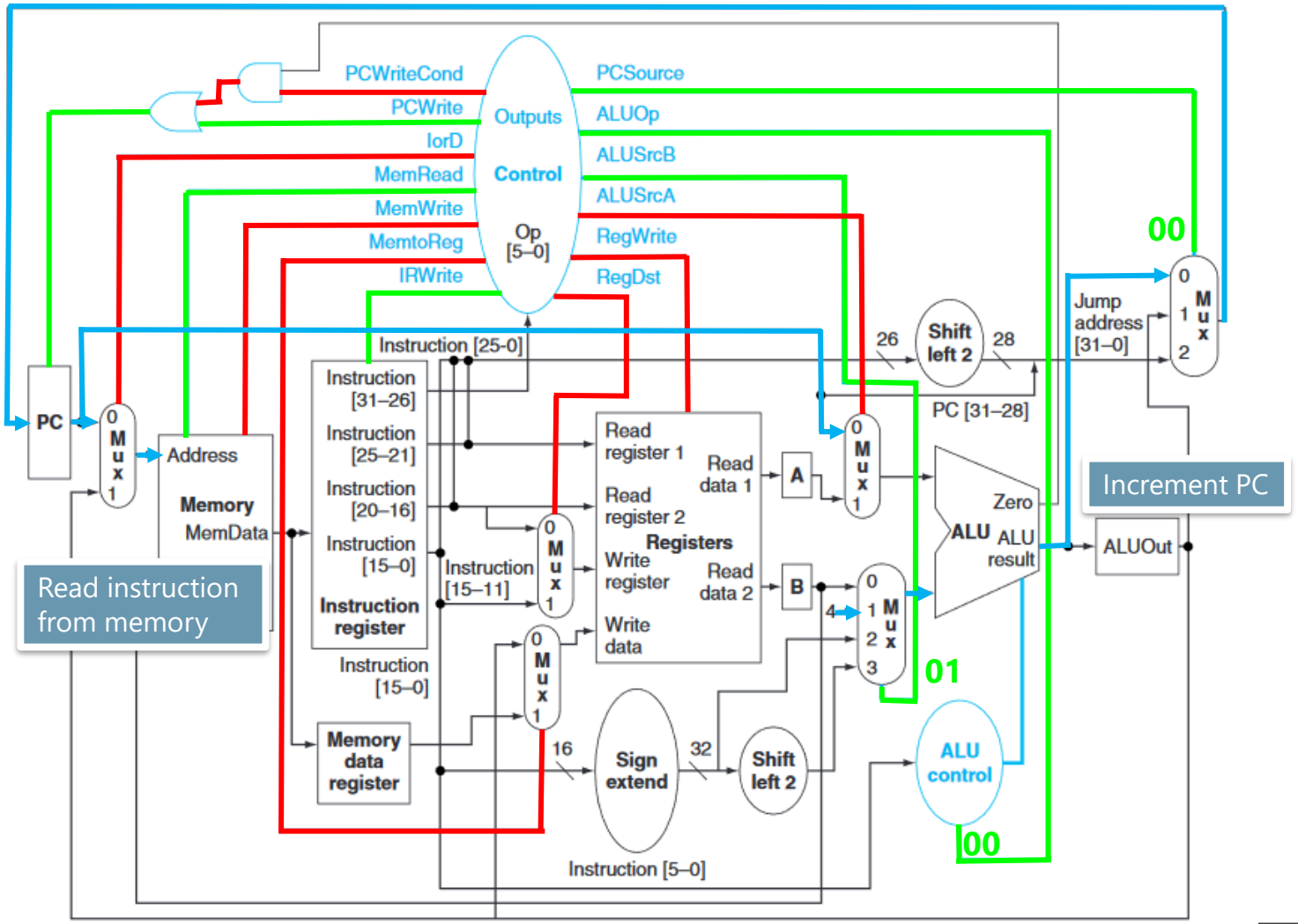
Signal	Effect
MemRead	Read from memory ➔ 0: Don't read; 1: Read
MemWrite	Write to memory ➔ 0: Don't write; 1: Write
ALUSelA	Select input of ALU input A ➔ 0: $pc ; 1: register A
RegDst	Select destination register ➔ 0: $rt (I-Type); 1: $rd (R-Type)
RegWrite	Register-file write enable ➔ 0: Don't write; 1: Write
MemToReg	Select data to write to the register-file➔ 0: ALU; 1: Memory
IorD	Select the source for the memory address to be accessed ➔ 0: PC (instruction); 1: ALU (lw/sw/lh/lhu/sh/lb/lbu/sb)
IRWrite	Instruction register write enable ➔ 0: Don't write; 1: Write
PCWrite	PC unconditional write enable (jumps/PC=PC+4) ➔ 0: No effect; 1: Jump
PCWriteCond	PC conditional write enable (branches) ➔ 0: No effect; 1: Jump conditionally

(More) signals

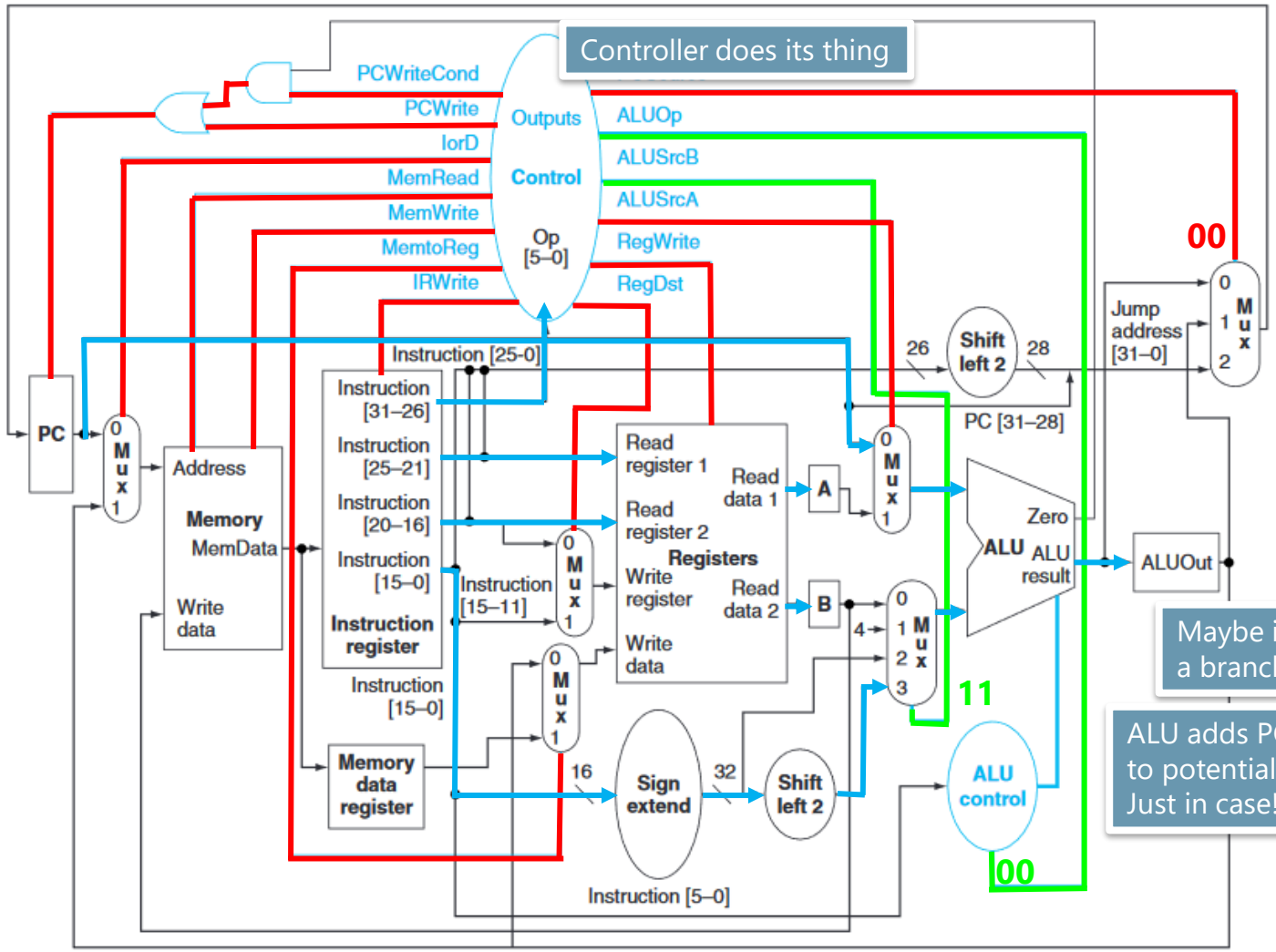
Signal		Effect
ALUSelB	00	ALU input B comes from $rt
	01	ALU input B is the constant 4
	10	ALU input B comes from the Immediate field
	11	ALU input B comes from the Immediate field shifted left 2 (branch)
ALUOp	00	ALU does an addition
	01	ALU does an subtraction
	10	ALU behaviour depends on the function field (R-Type)
PCSource	00	PC is updated with the result of the ALU (Fetch: PC+4)
	01	PC is updated with the result of the ALU (PC = PC + branch offset)
	10	PC is updated with the Jump target (jump)

Fetch and decode

Fetch Instruction

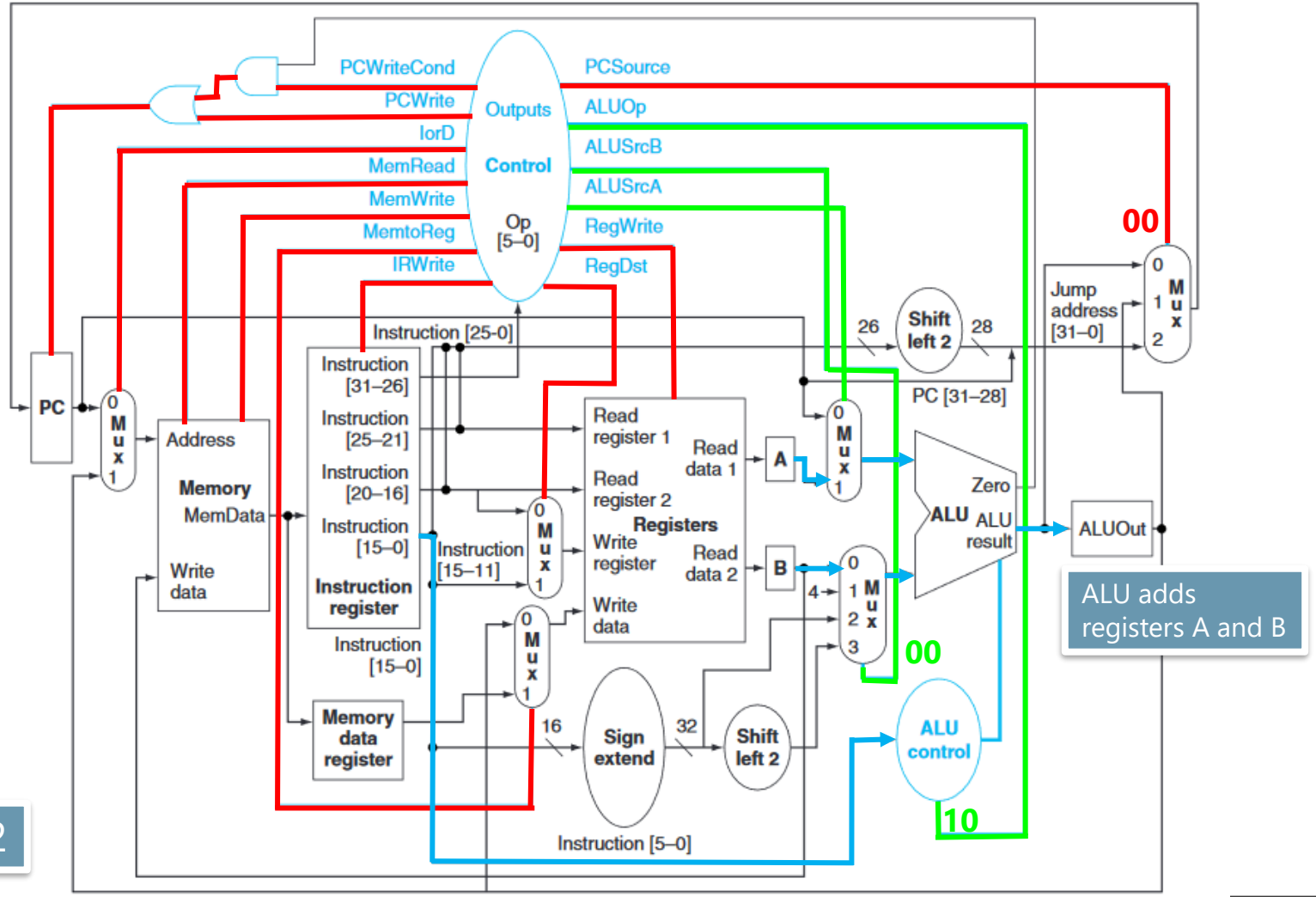


Decode Instruction



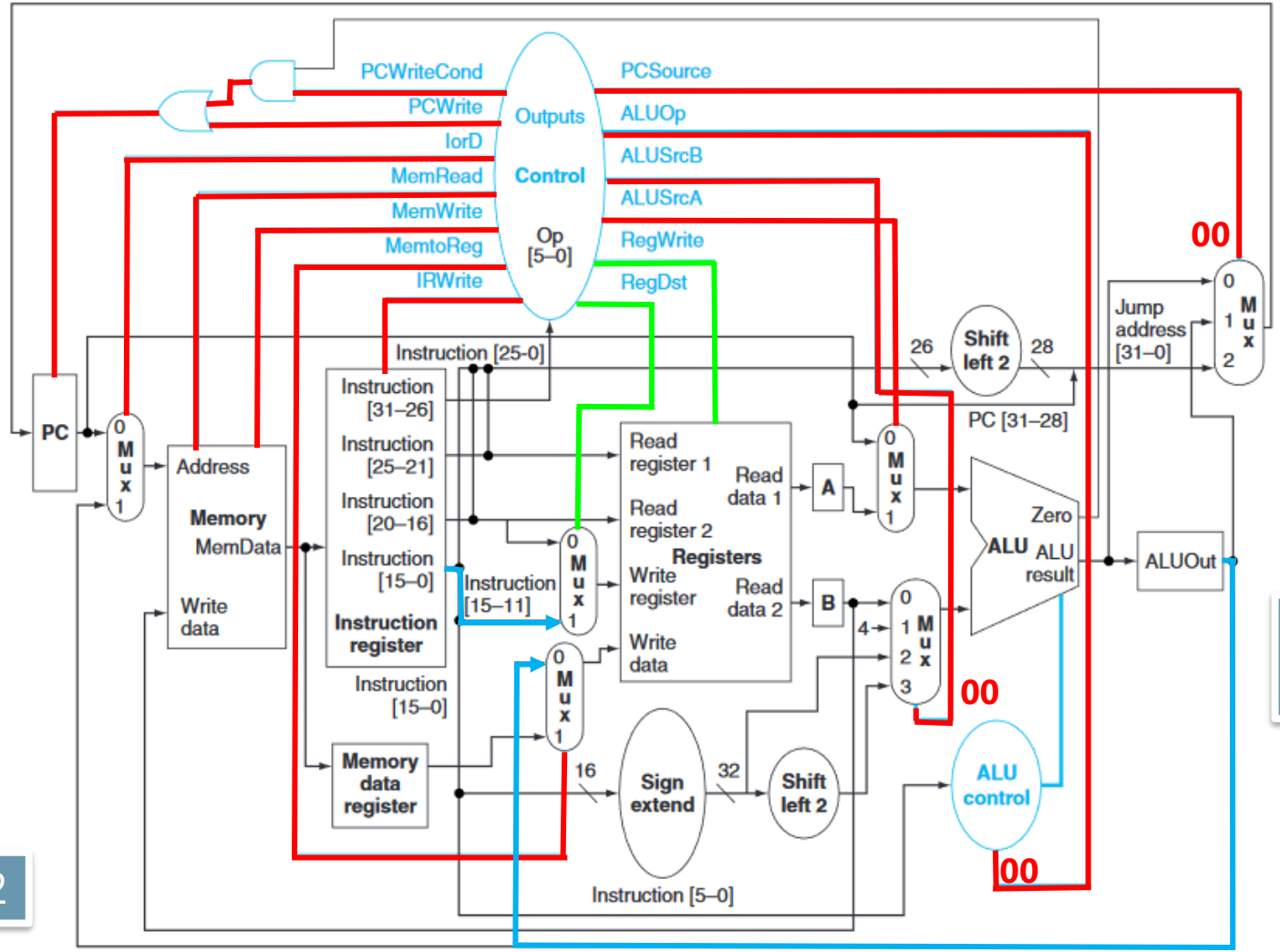
R-Type

Execute Instruction



add s0, s1, s2

Write Back

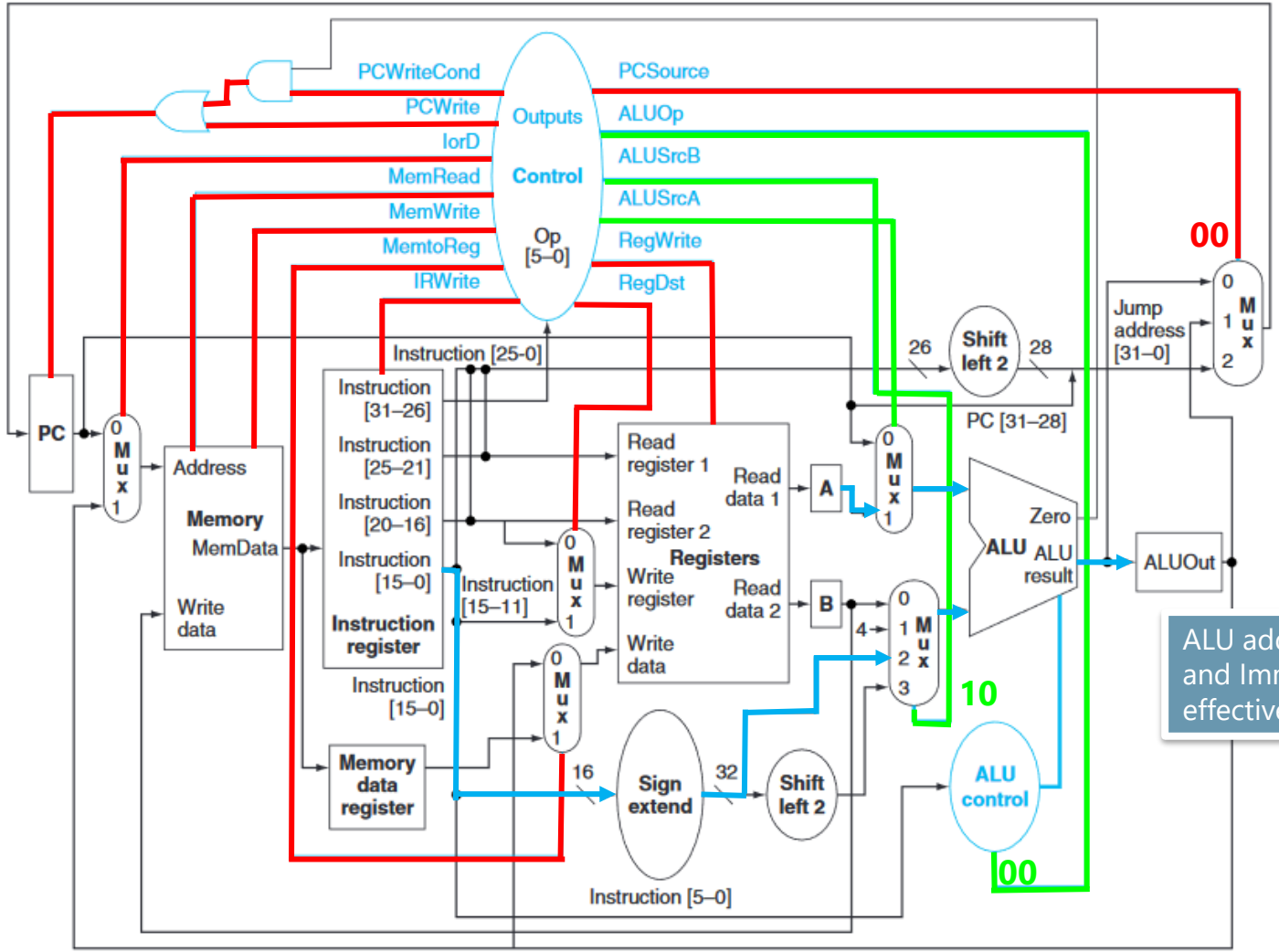


ALU result is written back into the Register File

add s0, s1, s2

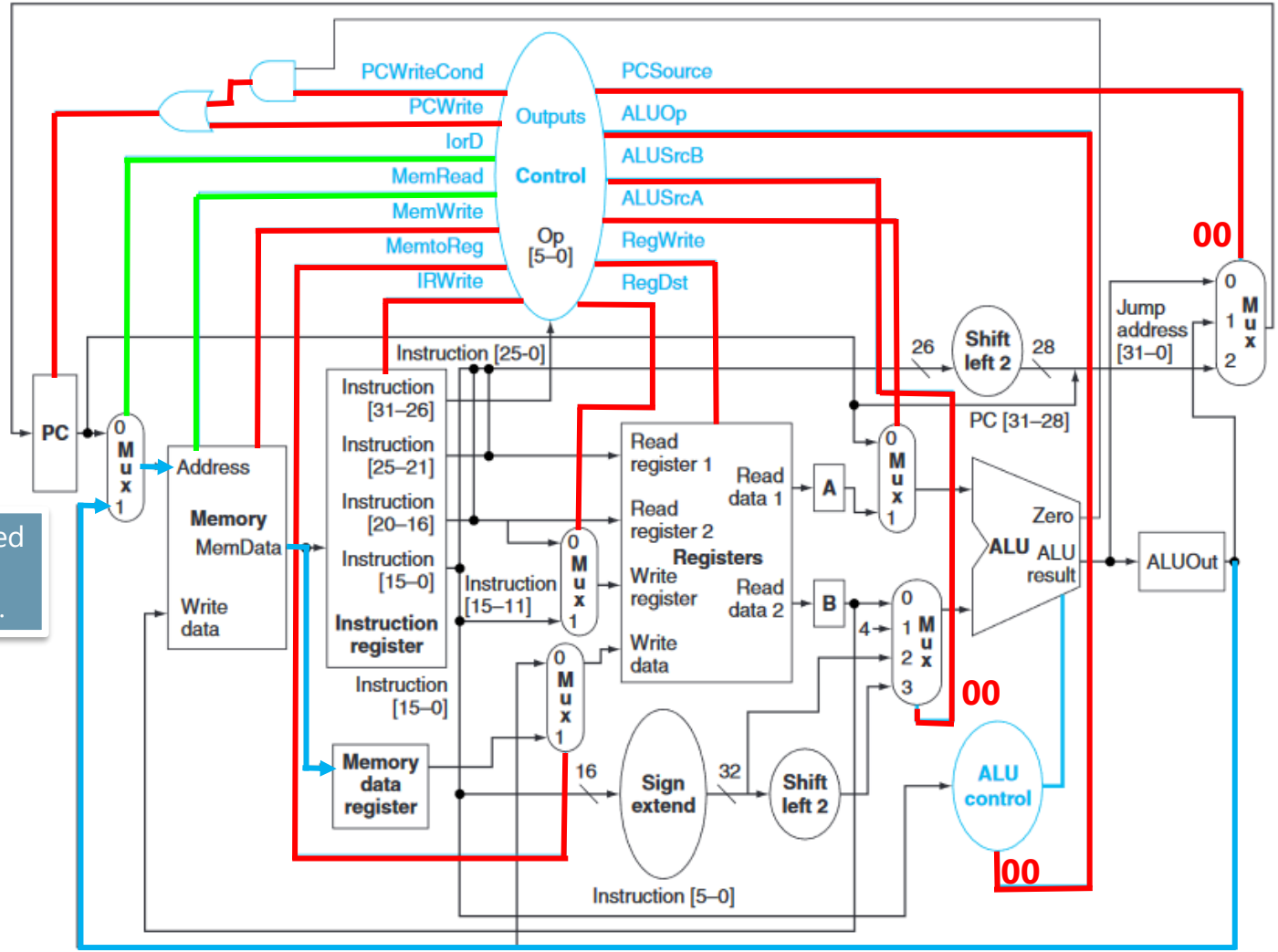
I-Type
lw

Execute Instruction



lw s0, 4(s1)

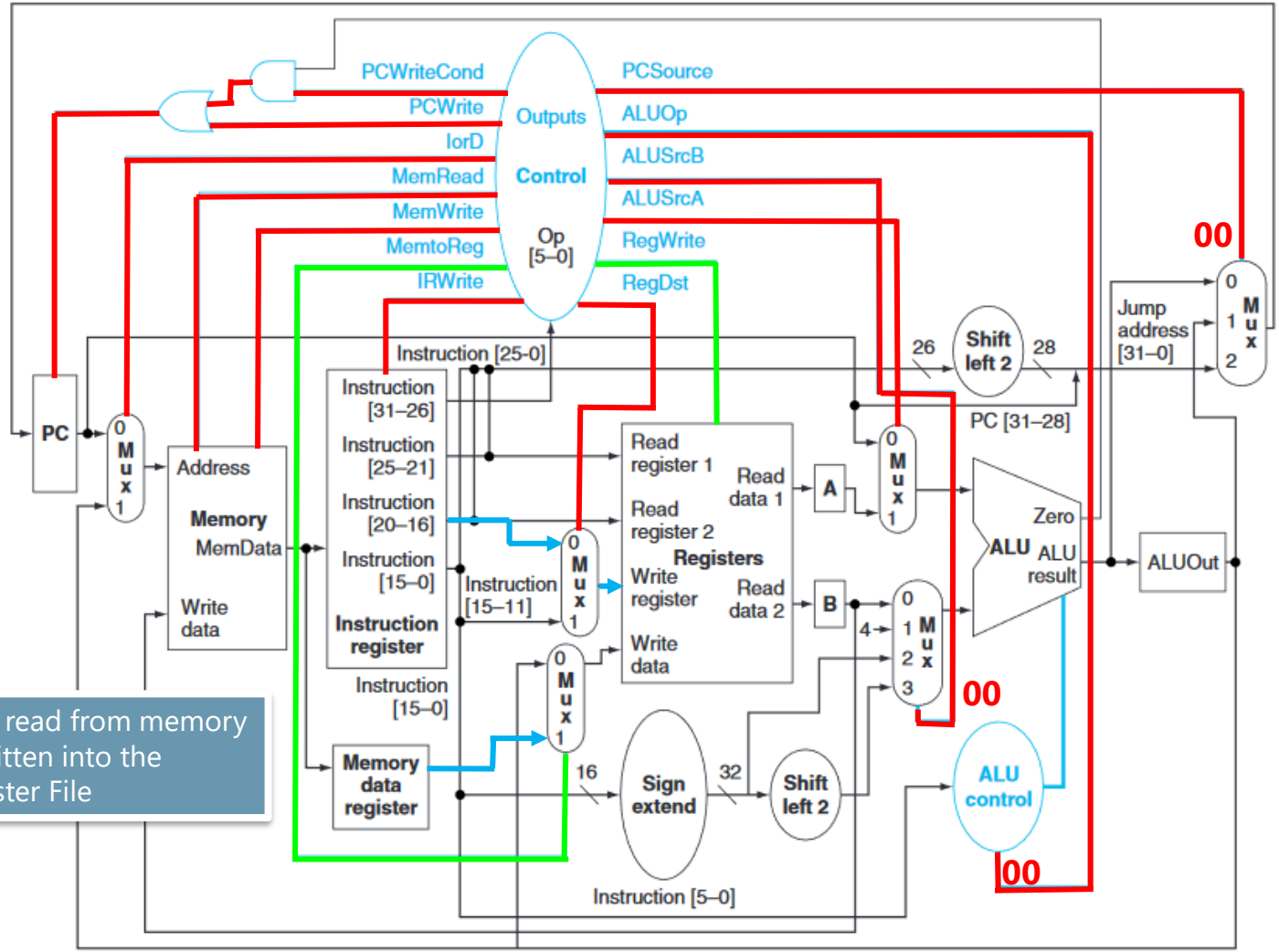
Memory access



ALU result is used as the address. Memory is read.

lw s0, 4(s1)

Write back

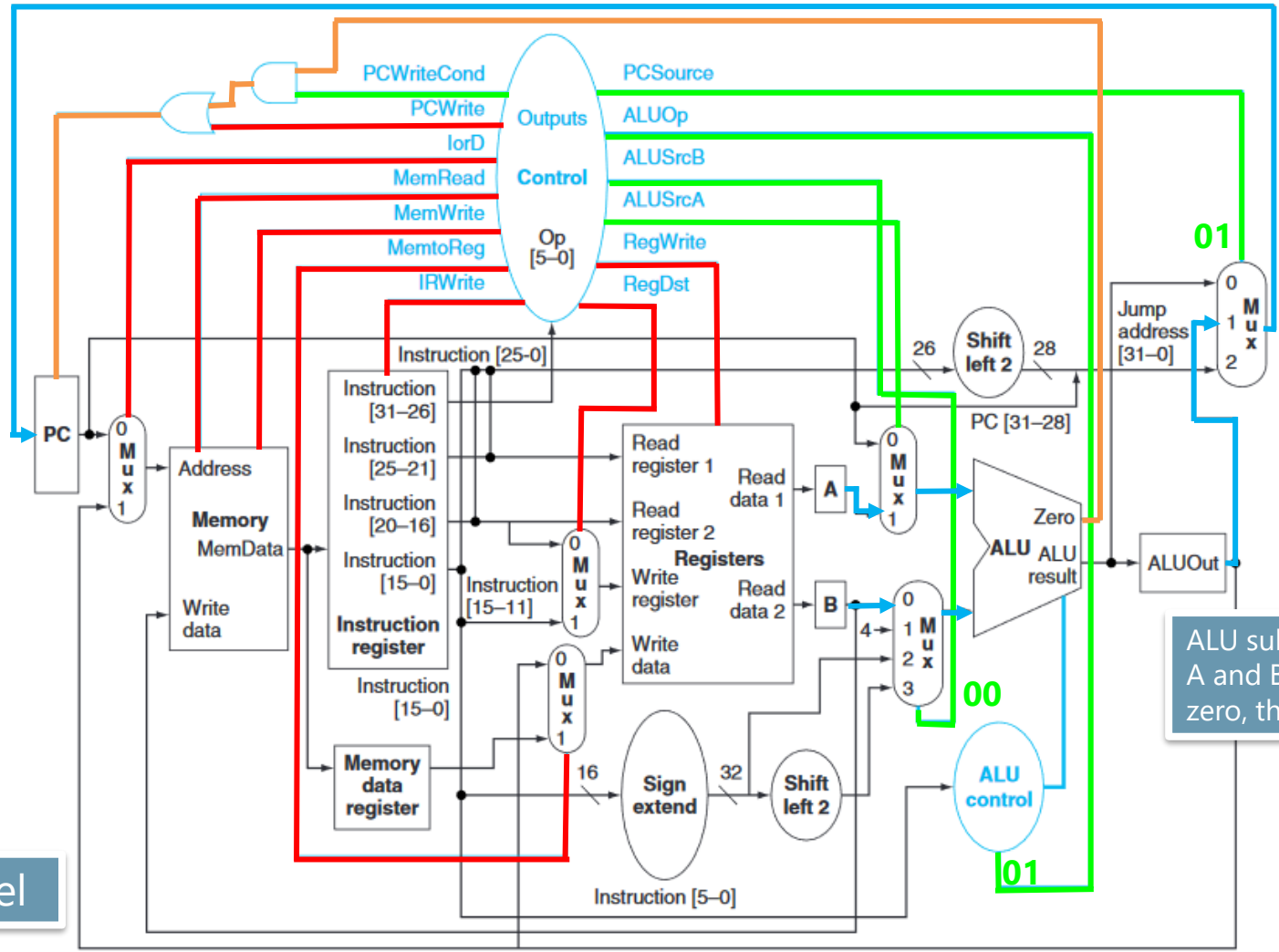


Data read from memory is written into the Register File

lw s0, 4(s1)

I-Type
beq

Execute Instruction – Branch conclusion

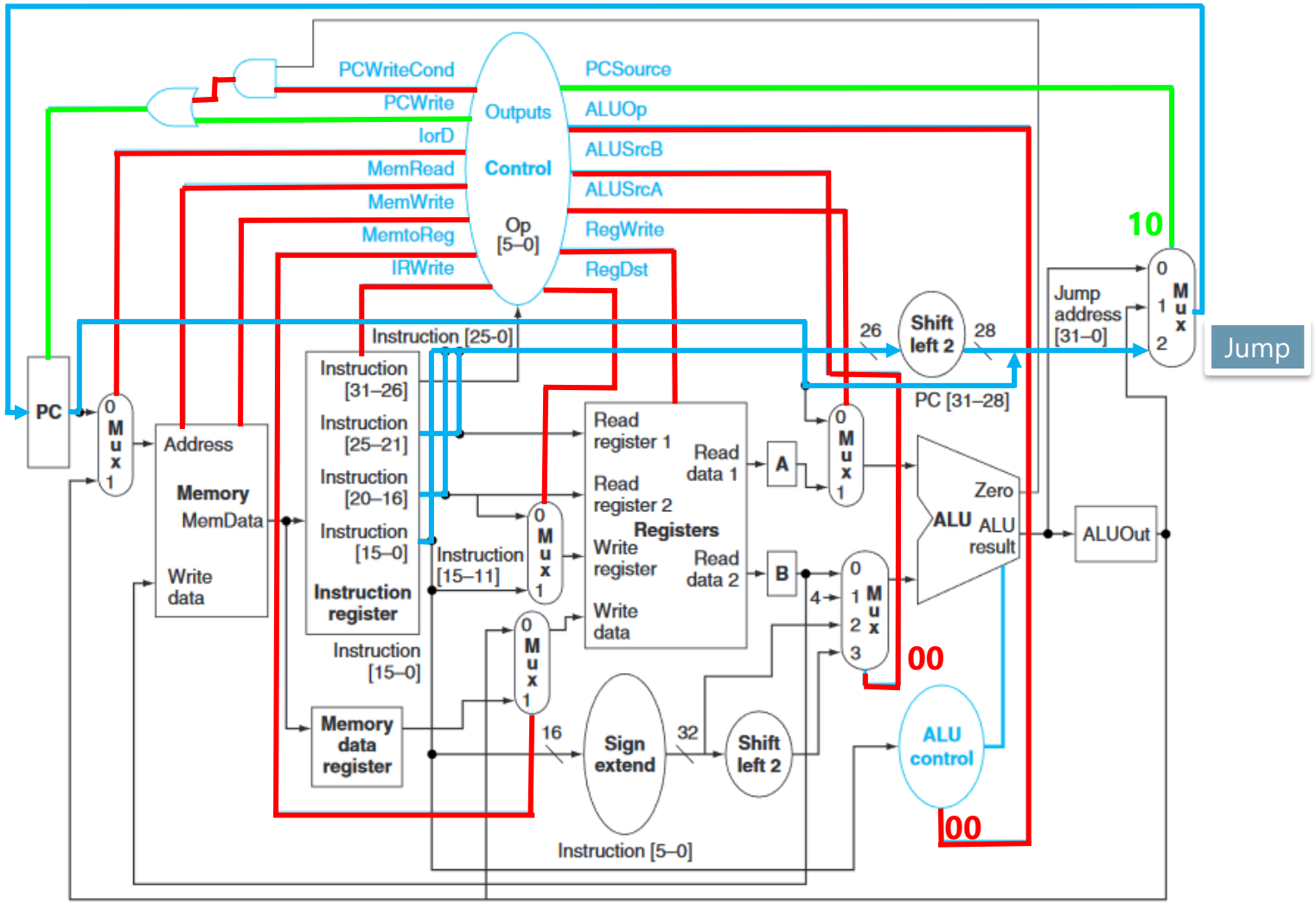


ALU subtracts registers A and B if the result is zero, then branch

beq s0, s1, label

J-Type
jump

Execute Instruction – Jump



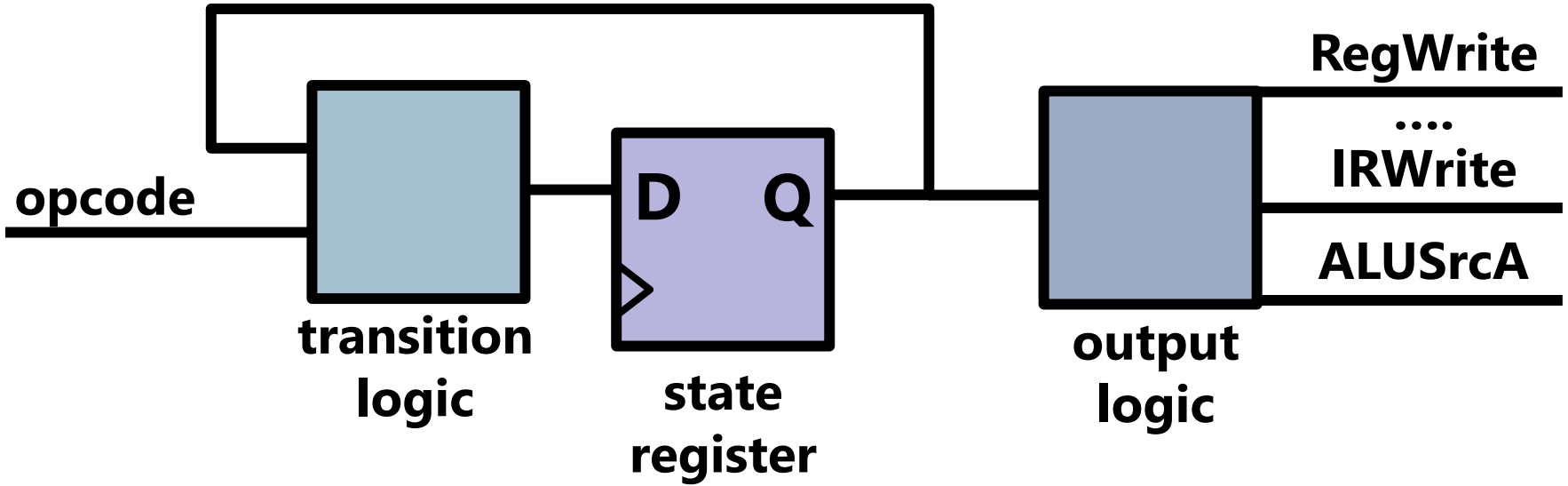
Multicycle Control

- How are **control signals generated** on each cycle?
  - Single-cycle: Signals don't change during each instruction
    - Combinational circuit
  - Multi-cycle: Signals change during each instruction
    - Different signals each clock cycle
      Sequential circuit → Needs to remember what it did before

- What are the **transitions** between cycles?
  - (i.e., what happens next?)

- How to describe this behaviour?
  - State machine!

Tick-tock

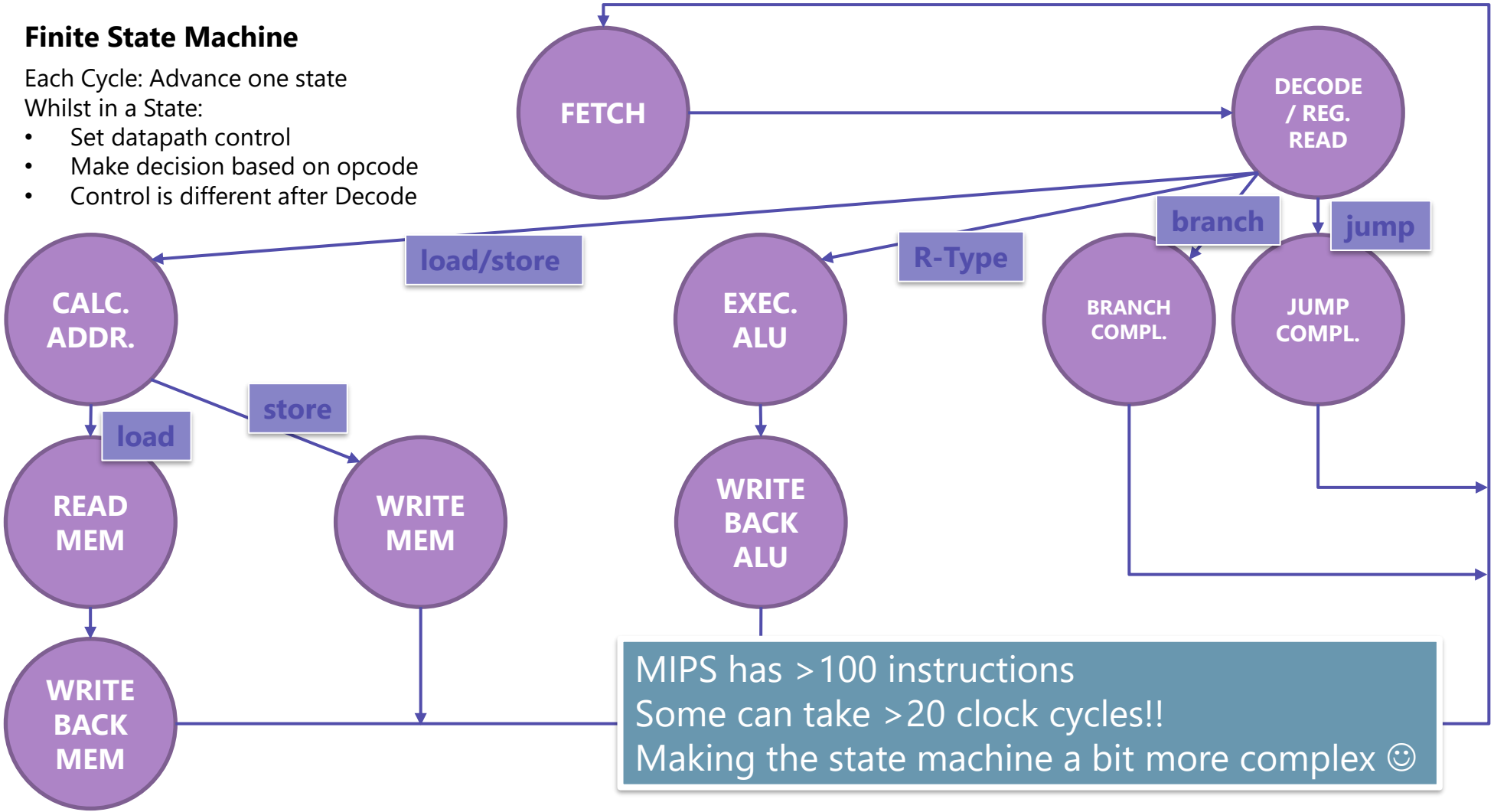

opcode
transition
logic
D
Q
state
register
output
logic
RegWrite
....
IRWrite
ALUSrcA

Multicycle Control

Finite State Machine

Each Cycle: Advance one state
Whilst in a State:

- Set datapath control
- Make decision based on opcode
- Control is different after Decode



MIPS has >100 instructions
Some can take >20 clock cycles!!
Making the state machine a bit more complex ☺

Performance

The layman's understanding

- your ancient computer takes 30 seconds to open the browser
- you get a new computer. it opens the browser in 3 seconds.
  - **which computer is faster?**





yeah but this is computer *science*, not computer *guessing*.

Old stuff!!

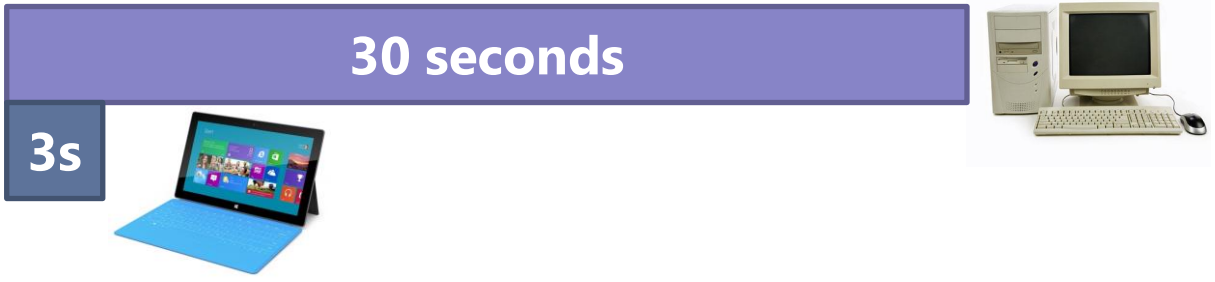
- you wanna copy a CD as many times as you can in 12 minutes
- both the PC and this... thing take **4 minutes** to copy
- which device will make **more copies** in **12 minutes? why?**
  - numbers usually mean we're getting more science-y, right?



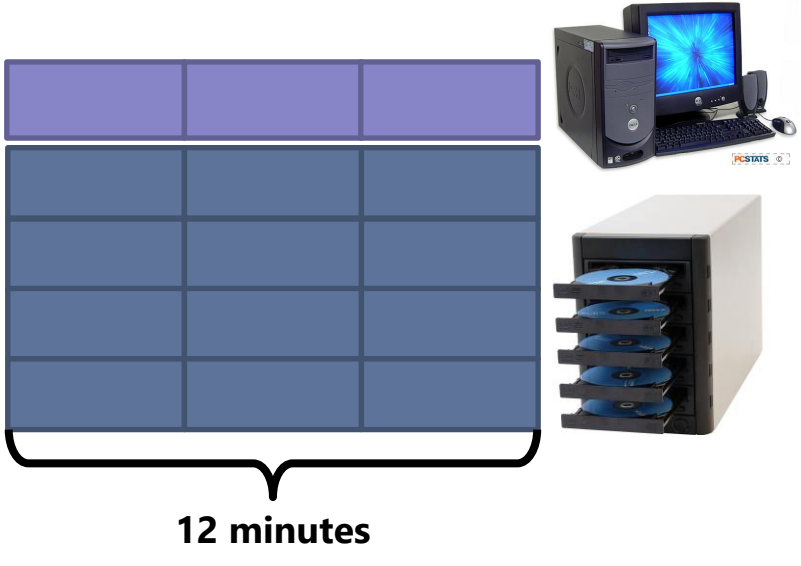
Response time and throughput

- **response time** is the length of time **from start to finish**
- **throughput** is the **amount of work** you can do in a span of time

response time
(time per task)



throughput
(tasks per time)



they're not quite *reciprocals* of each other; their relationship is a little more complex

Response time can improve throughput!

- you put a brand new 52X CD burner in your sweet Dell. it burns a CD in only 2 minutes.



4min	4min	4min

2min	2min	2min	2min	2min	2min

- the response time for a single CD burn is improved, but this also causes our **throughput** to double!
- this is because the **measurement period** stayed the same (12min)

Throughput can improve response time!

- someone wants you to make them **20 copies** ASAP
- how long would it take with **one** CD duplicator?
- how long would it take with **two**?



4min	4min	4min	4min	4min
4min	4min	4min	4min	4min
4min	4min	4min	4min	4min
4min	4min	4min	4min	4min
4min	4min			
4min	4min			
4min	4min			
4min	4min			

with one: 20min

with two: 12min

this is because the **workload** stayed the same (20 copies)

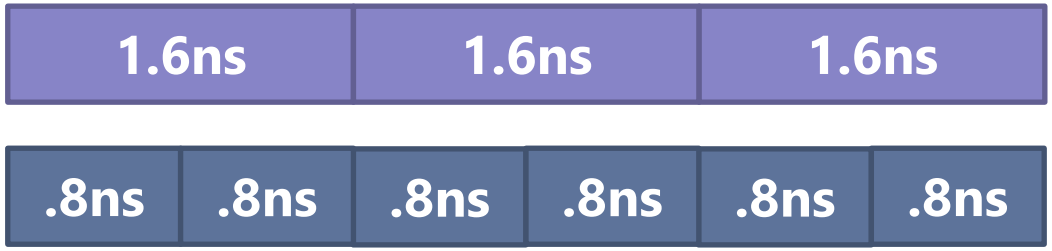
Applying it to a CPU

- the CPU's job is to run instructions. so we could...
  - do each instruction *faster* (i.e. reduce **latency**)
  - do *more instructions at once* (i.e. increase **throughput**)
- for a long time, we did the former...
  - clock speeds increased by 2 orders of magnitude since ~1990
    - I had a 33MHz! 80486 PC
- but then we hit a wall.
  - and that's when multi-core CPUs became common.

Reducing latency

- you put a new Pentium 4 in your sweet Dell. it executes a single instruction in only 0.8 nanoseconds.





if each instruction takes only 0.8ns, that means 0.8ns between clock pulses. **How fast is the clock running?**

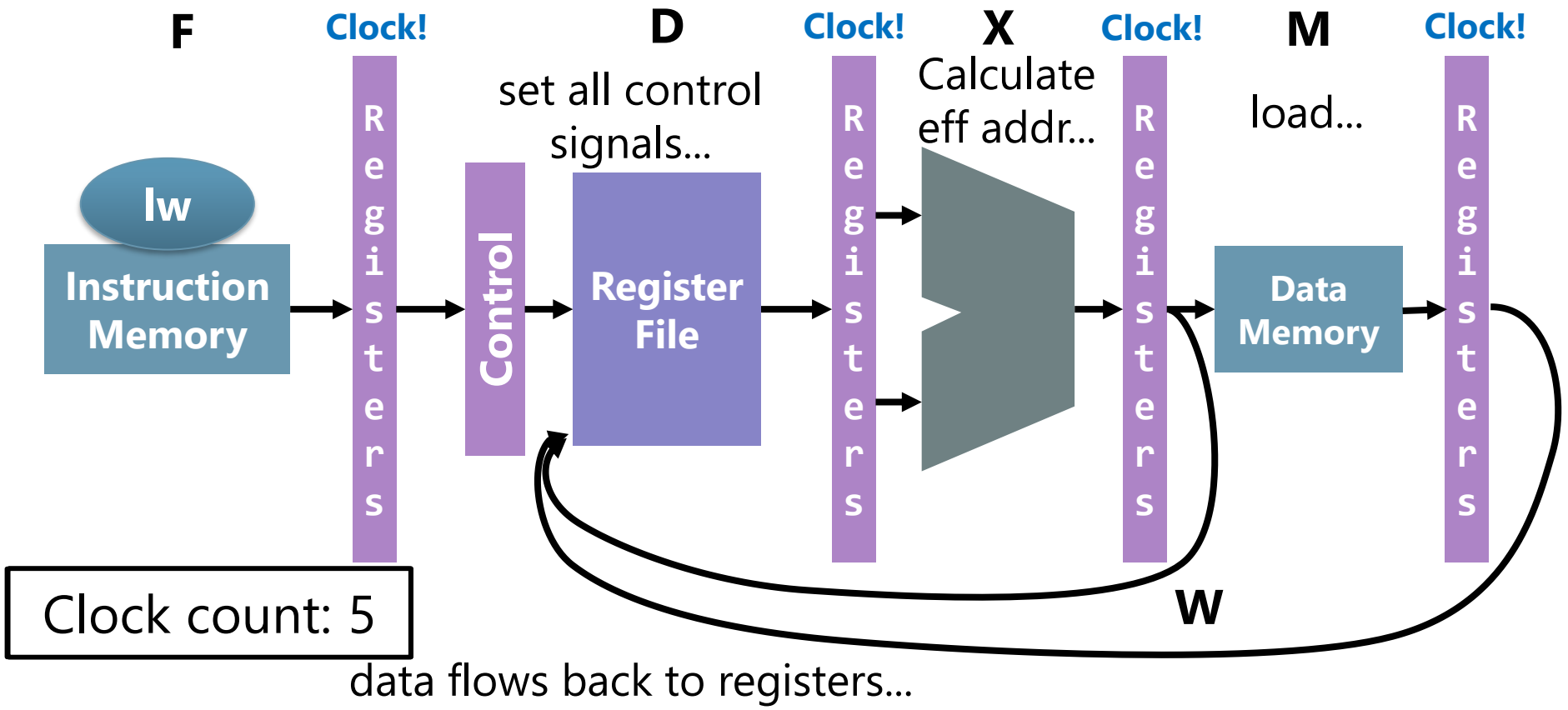
$$\frac{1}{0.8 \times 10^{-9} s} = 1.25 \times 10^{9} Hz$$

$$= 1.25\ GHz$$

But is this efficient?

Multi-cycle lw (animated)

- Let's watch a **lw** instruction flow through the datapath!



Real-World Analysis

Another example

```
.data
A:      .word 10,20,30,40,50,60,70,80,90,100
B:      .word 0,0,0,0,0,0,0,0,0,0
.text
        la      $s0,A           # address of A
        li      $s1,10          # A[i] * 10
        li      $s2,10          # iteration
loop:   lw      $t0,0($s0)      # read A[i]
        mul     $t0,$t0,$s1     # $t0=A[i]*10
        sw      $t0,40($s0)     # update A[i]
        addi    $s0,$s0,4       # next element
        addi    $s2,$s2,-1      # dec iteration
        bne     $s2,$0,loop     # done?
        li      $v0,10          # exit syscall
        syscall                 # syscall
```

Another example

```
.data
A:      .word 10,20,30,40,50,60,70,80,90,100
B:      .word 0,0,0,0,0,0,0,0,0,0
.text                                   #               instr. count
        la      $s0,A                   # address of A  2
        li      $s1,10                  # A[i] * 10     1
        li      $s2,10                  # iteration     1
loop:   lw      $t0,0($s0)              # read A[i]     10
        mul     $t0,$t0,$s1             # $t0=A[i]*10   10
        sw      $t0,40($s0)             # update A[i]   10
        addi    $s0,$s0,4               # next element  10
        addi    $s2,$s2,-1              # dec iteration 10
        bne     $s2,$0,loop             # done?         10
        li      $v0,10                  # exit syscall  1
        syscall                         # syscall       1
```

Another example

- Let's analyze the program.

- How much time with a Single-Cycle design?
  1. What's the clock length?
     - Assume a clock speed of 100 MHz (cycle length is 1/100MHz, or 10ns)
  2. What's the cycles per instruction?
     - CPI: 1 (single-cycle! yay!)

- Thus this program executes in this much time:
  - 66 instructions * 1 CPI * 10 ns = 660ns

Another example

- How much time with a Multicycle design?
  1. We need to know the clock length (what is the clock speed?)
     Let's just assume the ideal efficiency: divide the clock length by 5.
     - (Because we said loads, the slowest, take 5 cycles while arithmetic takes 4)
     - AKA multiply the clock speed by 5 to 500 MHz
     - Either way, the clock length is 10ns / 5 = 2ns

  2. What's the CPI?
     - We need to know how many of each type there are. Look at the program.
     - Arithmetic: 36, Branches: 10, Loads: 10, Stores: 10
     - There's a different # of cycles per each type (4, 3, 5, and 4 respectively)

- Thus this program executes in this much time:
  - 36*4*2ns + 10*3*2ns + 10*5*2ns + 10*4*2ns = 528ns

- Multicycle, here, let's us improve our program execution from 660ns to 528ns (20% reduction!)

doo-doo-doo doo-doo-doo-doo-doo

- not bad! I guess?
  I mean, we increased the clock speed by a factor of 5...
  and we only got 20% reduction on execution time
  if our CPI were also close to 1, it'd be 10 times as fast as the single-cycle machine...
  - How do we increase throughput...
  - HMMMMMMMMMMMMMMMMMMMMMMM...